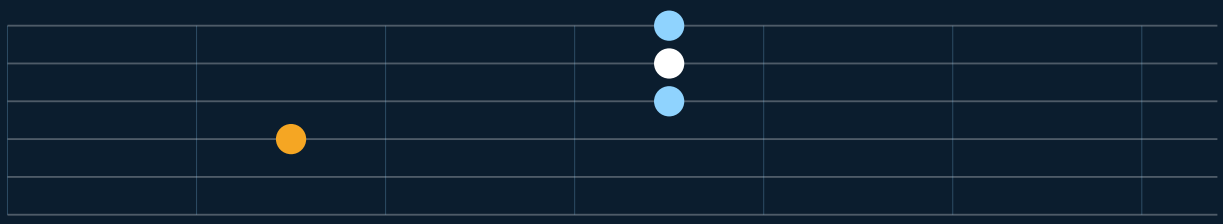
DROP ROUTE

Documento de Diseño Técnico del Software

Voicings, rutas II–V–I y patrones de escala
Base de desarrollo para Claude Code



Autor del proyecto: Jesús Quevedo Rodríguez

Revisión parcial: versión 1.1.0 y patrones de escala · 29/10/2026
Repositorio: a01653/Drop_Route
Estado: aplicación de estudio y práctica

Índice

1 Objetivo y alcance **2**

2 Arquitectura general **2**

3 Tecnologías y dependencias **3**

4 Estructura actual **4**

5 Componentes previstos **5**

6 Estado y persistencia **5**

6.1 Progreso local (DR-006) 6

7 Modelo musical **8**

8 Catálogo y construcción de voicings **8**

9 Rutas y conducción de voces **8**

10 Patrones de escala (DR-013) **9**

11 Preguntas y corrección **10**

12 Móvil y accesibilidad **11**

13 Offline e instalación **11**

14 Documentación y Claude **12**

15 Flujo de datos **13**

16 Pruebas y resultados reales **13**

17 Construcción y publicación **15**

18 Limitaciones actuales **17**

19 Evolución **18**

20 Apéndice: evidencia y límites **18**

1 Objetivo y alcance

Drop Route servirá para memorizar voicings, inversiones y grados y practicar rutas II-V-I concretas. Ordenador y móvil. Primera etapa en C, con la apariencia de nuestras tarjetas. Desde DR-013 incluye los cinco patrones de cuatro escalas, en estudio y en dos juegos con tonalidad elegible.

Se preparan fuentes e instrucciones para que Claude Code implemente cada etapa mediante órdenes de trabajo. No necesita acceder al historial de esta conversación ni recibir las plantillas de nuevo.

El 13/10/2026 el usuario decidió mantener este Markdown actualizado durante DR-001 a DR-006 y concentrar la generación LaTeX/PDF, los manuales completos, las capturas y la revisión visual documental en DR-007. Los derivados existentes son la última edición generada y pueden quedar pendientes respecto a este Markdown.

2 Arquitectura general

Aplicación cliente, sin backend.

Capa	Responsabilidad
Datos	Posiciones y rutas procedentes de la plantilla; patrones de escala transcritos de una fuente publicada.
Núcleo musical	Alturas físicas, grados, inversiones y solución de ruta.
Sesión	Pregunta, selección, evaluación y transición.
Progreso	Contadores, configuración guardada, esquema versionado, migración y reparación.
Presentación	Tarjeta SVG, selectores, fichas, resultados, avisos y registro del service worker.

La geometría no decide la respuesta y los componentes no duplican el catálogo. Están implementados la extracción y el núcleo de auditoría de la base y, en `src/`, cinco capas:

Capa	Contenido
`src/data`	Tipos y acceso a `data/catalog.json`, `data/routes.json` y `data/scales.json`, que siguen siendo la única definición de datos.
`src/music`	Altura física (MIDI), inversión por bajo real y, en `scalePatterns.ts`, patrones de escala colocados en una tonalidad y su orden de grave a agudo. Funciones puras.
`src/session`	Menú, preguntas de reconocimiento, de ruta y de los dos juegos de escalas (`scales.ts`, `scaleOrder.ts` y `scaleSession.ts`), respuesta, corrección por campos, banco de grados y reductor de acciones, con semilla reproducible. Sin React.

Capa	Contenido
`src/progress`	Contadores por juego, set y dificultad o modo y tramo; reductor de la aplicación que cuenta un intento por transición; esquema guardado, migración, reparación y copia; almacén (`createAppStore`) sobre un `KeyValueStore` mínimo; resumen. Sin React ni acceso directo a `localStorage`.
`src/ui`	Menú y pantallas de juego (`game/`, con el panel de progreso y las pantallas de patrones y de ordenar), modo estudio (`study/`), renderizador de tarjetas SVG (`card/`, con la tarjeta de patrón de seis cuerdas), pantalla de revisión (`review/`), avisos (`Notices.tsx`), adaptador de `localStorage` (`browserStorage.ts`) y registro del service worker (`pwa/`).

La dependencia permitida es ui, progress, session, music, data. ESLint rechaza que una capa importe otra superior y que data, music, session o progress usen React, de modo que pueden probarse sin DOM. Un test de capas de `src/progress` lo comprueba. Solo `src/ui/browserStorage.ts` toca `localStorage`.

3 Tecnologías y dependencias

Node.js 22.13 o superior (campo engines; la CI usa Node 24 según .nvmrc) ejecuta tests, auditorías y herramientas. Python 3.10 o superior extrae las constantes literales y genera el DTS. Lo localiza el lanzador `scripts/run-python.mjs`: prueba la variable PYTHON y después py -3, python y python3 en Windows, o python3 y python en otros sistemas. Si no hay una versión válida, el error lista lo probado y explica cómo resolverlo. Los tests y auditorías de la base con node:test no dependen de paquetes externos.

DR-001 fijó estas versiones en el lockfile de npm. Instalación: `npm ci --ignore-scripts`. No se copiaron dependencias innecesarias de Mástil.

Paquete	Versión	Nota
react, react-dom	19.3.0	Única dependencia de ejecución.
typescript	6.0.3	Se fija la serie 6.0 porque typescript-eslint 8.71.2 exige una versión inferior a 6.1. La 7.0.2 es posterior y no es compatible.
vite, @vitejs/plugin-react	8.3.4, 6.1.2	Build y servidor de desarrollo.
vitest	5.0.4	Tests unitarios y de capas, sin DOM.
eslint, @eslint/js	10.12.0, 10.0.1	Lint y límites entre capas.
typescript-eslint	8.71.2	Reglas para TypeScript.
eslint-plugin-react-hooks, eslint-plugin-react-refresh, globals	7.1.1, 0.5.7, 17.13.0	Reglas de React y entornos.

Paquete	Versión	Nota
@playwright/test	1.64.0	E2E con Chromium.
@types/node, @types/react, @types/react-dom	24.19.2, 19.3.0, 19.3.0	Tipos.

Vite 8 y Vitest 5 exigen Node 22.12 y ESLint 10, 22.13; de ahí el mínimo de engines. Playwright necesita descargar el navegador con `npx playwright install chromium`; ese paso queda fuera de npm ci.

Pandoc y LuaLaTeX completo o XeLaTeX generan este documento. El estilo utiliza la paleta azul oscuro y familias tipográficas de la documentación de Mástil, con equivalentes si faltan TeX Gyre. Idioma español y sin partición de palabras al final de línea.

4 Estructura actual

Ruta	Contenido
`CLAUDE.md` / `AGENTS.md`	Reglas permanentes y flujo de trabajo.
`data/catalog.json`	Sets, fórmulas, afinación y 124 formas.
`data/routes.json`	Dieciséis rutas II-V-I completas.
`data/scales.json`	Veinte patrones de escala en C (cuatro escalas por cinco formas), con la URL y el SHA-256 del diagrama fuente de cada uno.
references/	Tres fuentes originales y sus hashes.
scripts/	Extracción, auditoría (también de los patrones de escala, en `scripts/lib/scale-validation.mjs`), generación documental y lanzador de Python.
tests/	Pruebas con node:test, conservadas aparte: las de la base musical, el utillaje, la política documental y el icono.
`src/`	Capas `data`, `music`, `session`, `progress` y `ui`, `main.tsx` y los tests Vitest junto a cada capa.
`e2e/`	Playwright: humo, tarjetas, juegos, comparación con la plantilla, progreso, sin conexión y actualización de la PWA.
`assets/icon/`	Icono: original SVG editable, variante maskable, exportaciones PNG y su manifiesto con hashes.
`public/`	Favicon (copia idéntica del original del icono), `manifest.webmanifest` e `icons/` con los PNG de la PWA.
`scripts/pwa/`, `scripts/lib/pwa-build.mjs`, `scripts/pwa-plugin.mjs`	Plantilla del service worker, generador de su lista de recursos y versión, y plugin de Vite que lo ejecuta al terminar el build.
`index.html` y configuración	Vite, Vitest, Playwright, ESLint y dos `tsconfig` (aplicación y herramientas).
docs/	Requisitos, decisiones, tareas, validación y DTS.
`FILES_SHA256.json`	Instantánea histórica de los hashes de la base inicial del 13/10/2026, previa a DR-001. No es el estado actual ni se actualiza.
`RECOVERY_STATUS.json`	Instantánea histórica del estado de la reconstrucción, con sus cifras de entonces. No se actualiza.

Ruta	Contenido
.github/workflows/	Validación de la base y trabajo de aplicación, preparados para GitHub.

`npm run build` genera `dist/`, que Git ignora. El build solo incluye lo que importa `src/` y lo de `public/`: ni las referencias ni la plantilla HTML. Al terminar, el plugin escribe `dist/sw.js`. No hay flujo de despliegue ni alojamiento elegido; para publicar bajo una subruta, véase «Construcción y publicación».

5 Componentes previstos

Componente	Responsabilidad
Configuración	Juego, Drop, set, dificultad y modo de ruta.
Tarjeta	Cuerdas, trastes, notas y estado neutro/solución.
Respuesta	Cuatro botones o dos desplegables.
Grados	Cuatro destinos, arrastre, dos toques y teclado.
Rutas	Partida fija y una candidata visible.
Corrección	Resultado por campos y avance explícito.

De esta tabla existen la Configuración (el menú), la Tarjeta (neutra y de solución), la Respuesta (cuatro botones o dos desplegables), los Grados (cuatro destinos sobre la tarjeta, banco fijo, arrastre, dos toques y teclado) y la Corrección (resultado por campos y avance explícito). Las Rutas existen desde DR-005 (partida fija, una candidata visible, selector 1 a 4, verificación exacta y comparación). Las pantallas actuales son el menú, la pregunta de reconocimiento, la de ruta y la de revisión `#/revision`, que no es un juego. El estado de la sesión vive en un almacén (`createAppStore`, `src/progress`) que `App` consume con `useSyncExternalStore`; el almacén aplica el reductor puro de la sesión (`reduceSession`) a través de `reduceApp`, que además cuenta el progreso. Las semillas las genera la interfaz y no el reductor. El menú muestra el panel «Tu progreso» y, si procede, los avisos de almacenamiento y de actualización.

La vista neutra solo recibe las posiciones de las notas y la de solución añade nombre, calidad y grados. Son tipos distintos, de modo que la neutra no puede dibujar ni etiquetar la solución. La geometría es una función pura, probada sin navegador. Las medidas, las ventanas de trastes, los colores y las fuentes están descritos en `docs/VISUAL.md`.

Compartir el renderizador conserva proporciones entre juegos. El icono se creó en DR-002 y está aprobado.

6 Estado y persistencia

Modelo: configuración, pregunta, pregunta verificada y regreso al menú. Cada pregunta identifica datos y guarda sus opciones y orden.

Seleccionar no evalúa. Verificar exige respuesta completa y produce un único resultado. Después se bloquea la respuesta. Siguiente genera otra pregunta y Salir vuelve al menú.

La corrección no sobrescribe lo que contestó el usuario. Si falla el V de una ruta, el paso V-I debe partir del V correcto.

La sesión implementa ese modelo con tres fases: `menu`, `pregunta` y `verificada`. Seleccionar cambia la respuesta pero no verifica ni puntúa. Verificar solo actúa con la respuesta completa del nivel, calcula el resultado una sola vez y bloquea la respuesta; verificar de nuevo, cambiar campos o mover grados después no hace nada. Siguiente abre otra pregunta con la misma configuración. En reconocimiento descarta todas las formas que ya salieron en esa ronda, hasta agotar las disponibles (actualmente 16 por set). Salir vuelve al menú y conserva la última configuración.

Desde 1.0.0, `RecognitionQuestion.roundVoicingIds` conserva las tarjetas ya mostradas en la ronda. `createQuestion` elige con igual probabilidad entre las pendientes; al agotarlas reinicia la ronda y excluye de su primera elección la tarjeta inmediatamente anterior, salvo que solo exista una. La lista no se muta y queda acotada al tamaño del set. Verificar conserva la misma pregunta y Siguiente recibe su historial, también en Medio y Difícil. El orden de los cuatro botones se sigue barajando por separado. La rotación es memoria de la partida: Salir y empezar de nuevo o recargar inicia otra; no se guarda en los contadores ni exige cambiar el esquema de persistencia. El selector de rutas no cambia.

`App` muestra una etiqueta de versión en todas sus pantallas, incluida la de estudio. Importa únicamente `version` de `package.json`, sin duplicar un número en los componentes ni restaurar el antiguo bloque informativo. La etiqueta se introdujo en 1.0.0; la corrección del desplazamiento en móvil corresponde a 1.0.1.

6.1 Progreso local (DR-006)

`reduceApp` (`src/progress/counting.ts`) llama al reductor de la sesión y, si el estado cambia, decide en la misma transición pura si cuenta un intento. Solo cuenta el paso de pregunta abierta a verificada, que la sesión permite una única vez: verificar de nuevo, seleccionar, cambiar de vista, Siguiente, Salir y renderizar dos veces no cuentan. Es acierto el acierto completo del nivel (todos los campos: acorde en fácil, acorde e inversión en medio y, en difícil, también los cuatro grados) o, en rutas, la candidata exacta de nuestra ruta.

Cada contador cuenta por separado: reconocimiento por set y dificultad (`recognition:<set>:<dificultad>`) y rutas por set, modo y tramo (`routes:<set>:<modo>:ii-v` o `v-i`). «II → V → I» no es un tramo: cada mitad suma en la suya. Los contadores posibles son solo los que existen en los datos: los siete sets del catálogo en tres dificultades y los sets con rutas, en sus modos y tramos. Un contador ausente equivale a cero intentos.

El documento guardado es un único texto JSON en `localStorage` bajo `drop-route:state`: `schemaVersion`, la configuración del menú (juego, Drop, set, dificultad, modo y tramo) y los contadores `{ attempts, correct }`. El orden es fijo, de modo que dos estados iguales dan el mismo texto y no se vuelve a escribir si no hay nada nuevo. `createAppStore` lee una sola vez al abrir y escribe el documento completo tras cada acción que cambie la configuración o los contadores. Al abrir se vuelve siempre al menú: una pregunta abierta no se reanuda, solo se conserva la configuración.

El esquema vigente es el 2 desde la 1.1.0 (DR-013). Es independiente de la versión del paquete: cambiar la versión de la aplicación no cambia el esquema, y cambiar el formato exige subir el esquema y añadir una migración. La cadena de migraciones (`MIGRATIONS`) convierte cada versión n en la n + 1. El esquema 0 es un fixture de prueba (`src/progress/fixtures/schema-0.json`): un formato inventado, con `counters` y pares `[intentos, aciertos]`, que no corresponde a ninguna versión pública, porque Drop Route no había publicado ningún

guardado. Existe para ejercitar la cadena con un caso real antes de que haga falta una migración verdadera. La migración 1 → 2 es la primera real: añade a la configuración la escala y la tonalidad de los juegos de escalas con sus valores por defecto (mayor y C), de modo que un documento correcto del esquema 1 no se lee como reparado, y conserva antes el original en `drop-route:state:v1`. Los contadores no cambian; los de escalas son claves nuevas: `scale-patterns:<escala>:<dificultad>` y `scale-order:<escala>`. La tonalidad no separa contadores.

Situación al abrir	Qué hace	Aviso
Sin nada guardado	Empieza de cero y escribe con el primer cambio.	Ninguno.
Documento válido del esquema vigente	Lo usa tal cual.	Ninguno.
Esquema anterior	Lo migra en memoria, guarda antes una copia del original en `drop-route:state:v<n>` y solo entonces escribe el documento actual. Si la copia falla, no lo reescribe y esa sesión no escribe.	Ninguno si todo va bien; «formato anterior» sin copia.
Contadores o configuración inválidos	Descarta lo inválido contador a contador y conserva el resto. Guarda una copia del original en `drop-route:state:damaged` antes de reescribir; sin copia, no reescribe y esa sesión no escribe.	«Reparado», con o sin copia.
Texto ilegible o sin versión de esquema válida	Empieza de cero tras guardar una copia en `drop-route:state:damaged`; si la copia falla, deja el dato dañado donde está y esa sesión no escribe.	«Dañado», con o sin copia.
Esquema posterior al vigente	No lo interpreta ni lo toca: esa sesión no escribe.	«Versión más reciente».
Almacenamiento bloqueado o no disponible	Juega en memoria.	«No deja guardar datos».
Escritura que falla (cuota llena o bloqueo posterior)	Sigue en memoria y reintenta con el siguiente cambio; si vuelve a escribir, el aviso desaparece.	«No se ha podido guardar».

Las dos copias son un único hueco cada una: una copia posterior sustituye a la anterior. La interfaz no permite recuperarlas; quedan en el almacenamiento del navegador. Un aviso es «solo en memoria» cuando lo que hay en la página no está guardado: no disponible, escritura fallida y las sesiones que no escriben para proteger un dato anterior. En ese caso recargar lo pierde, y la interfaz lo dice.

El panel «Tu progreso» del menú resume los contadores (totales de los cuatro juegos, los de la configuración elegida y el detalle plegado). No hay puntuación, rachas ni clasificaciones.

7 Modelo musical

Afinación estándar EADGBE. MIDI al aire, cuerdas 6 a 1: 40,45,50,55,59,64. Altura real igual a MIDI al aire más traste.

Calidades evaluables en C: maj7 (0,4,7,11), 7 (0,4,7,10), m7 (0,3,7,10) y m7(b5) (0,3,6,10). Grados reales, incluidas b3, b5 y b7.

Inversión por bajo real: raíz, tercera, quinta o séptima (sexta en las variantes C6). No deducirla del primer elemento de un array.

Cdim7 se conserva como referencia: 0,3,6,9 y bb7. La fuente rotula su séptima como °7. El acorde es simétrico y no se evalúa sin fijar raíz y contexto.

8 Catálogo y construcción de voicings

No se generan posiciones nuevas. Drop 2 tiene tres sets; Drop 3 y Drop 2+4, dos cada uno. Cuatro calidades y cuatro inversiones dan 112 posiciones evaluables. Hay otras doce disminuidas de referencia.

La auditoría verifica cuerda/traste, fórmula, grados, bajo y orden por Drop. La extracción analiza datos literales sin ejecutar JavaScript. --check confirma que los JSON coinciden con la fuente sin sobrescribir.

Mantener ventana corta de las tarjetas. Algunas formas abarcan cinco trastes: no recortar para imponer una ventana de tres.

9 Rutas y conducción de voces

Normalizadas solo Drop 2 sets 1 y 2, mayor y menor, cuatro inversiones iniciales: dieciséis rutas.

II	V	I
Fundamental	2ª	Fundamental
1ª	3ª	1ª
2ª	Fundamental	2ª
3ª	1ª	3ª

Mayor Dm7-G7-Cmaj7. Menor Dm7(b5)-G7-Cm7. Comparar con la ruta exacta del material. Otra inversión de G7 puede ser musicalmente válida y no corresponder a este ejercicio.

No aplicar un optimizador que sustituya estas rutas ni afirmar mínimo universal. C6 aparece como sustitución opcional en ciertos pasos de la fuente; no se convierte automáticamente en otro acorde evaluable.

En el modo estudio, `transposeRoute` deriva `anchorFret` del traste de la nota con grado `1` en el acorde II y lo asigna a copias de los tres pasos. No modifica `data/routes.json`. Al transponer, el ancla recibe el mismo desplazamiento común que las notas. La tarjeta de solución sombrea esa columna con el color del borde de la calidad y pone su número de traste en negrita; `windowForPosition` amplía el mástil si la columna queda fuera de su ventana habitual. La vista neutra no muestra el ancla.

10 Patrones de escala (DR-013)

Datos. `data/scales.json` contiene la mayor, la pentatónica mayor, la pentatónica menor y la menor natural, con cinco patrones cada una, en C. Se transcribieron de los diagramas de onlineguitarbooks.com (consulta del 29/10/2026), con una sola fuente para no mezclar digitaciones. Cada patrón guarda la URL y el SHA-256 del diagrama y la posición que indica la página (su traste más grave). Los diagramas no se copian al repositorio. Numeración acordada con el usuario: 1 = forma E, 2 = D, 3 = C, 4 = A y 5 = G, subiendo por el mástil; las menores usan la forma de su propio acorde menor (Em...Gm).

Validación. `scripts/lib/scale-validation.mjs` recalcula cada nota desde afinación y traste, sin fiarse de las etiquetas, y comprueba:

- grados y distancia exacta entre notas consecutivas, sin huecos ni repeticiones;
- dos notas por cuerda en las pentatónicas y dos o tres en las de siete notas;
- la posición de la fuente y la tónica en las cuerdas de cada forma CAGED;
- que la sucesión E-D-C-A-G recorra una octava y que cada caja pentatónica enlace con la siguiente;
- que cada pentatónica esté contenida en la forma homónima de su escala de siete notas.

Lo ejecutan `npm run audit:music` y `tests/scale-patterns.test.mjs`, con oráculos y mutaciones deliberadas.

Tonalidad. `patternInKey` desplaza todas las notas la misma cantidad hasta que el traste más grave queda entre 0 y 11; las cuerdas al aire se dibujan a la izquierda del cejuelo. El orden de grave a agudo es el del traste más grave en esa tonalidad: en las 48 combinaciones de escala y tonalidad es una rotación de 1-2-3-4-5 sin empates. Se usa la forma movible de la fuente; la digitación en posición abierta de algunas tonalidades no se incluye. Los juegos de voicings y rutas siguen en C.

Tarjeta. `ScalePatternCard` y `scaleLayout.ts` dibujan seis cuerdas, varias notas por cuerda y, si hace falta, el hueco de cuerdas al aire, con las proporciones, fuentes y colores de las tarjetas de voicings. En la solución, 1, 3, 5 y 7 (♭3, ♭7 o 6 según la escala) llevan los colores de grado y el resto de notas va en blanco. La vista neutra solo recibe posiciones. Las tarjetas que se comparan comparten columnas y hueco para verse a la misma escala. Sin trastes, no hay números ni cejuelo y el nombre accesible usa columnas relativas.

Juegos. El menú pide escala y tonalidad (una de doce o «Aleatoria») en lugar de Drop y set, que se conservan para volver a reconocimiento. Cambia de juego en las fases nuevas `escala`, `escala-verificada`, `escala-resumen`, `orden`, `orden-verificada` y `orden-resumen`, con el mismo historial que reconocimiento.

- Reconocer patrones sortea sin repetir dentro de la ronda: cinco patrones en una tonalidad o sesenta combinaciones con tonalidad aleatoria. Fácil pide el patrón; Medio, además, todas las tónicas; Difícil, todas las notas de los grados pedidos, con un banco fijo de siete (1, ♭3, 3, 5, 6, ♭7 y 7). Para verificar hacen falta el patrón y, en Medio y Difícil, una marca. Cada nota se compara con su grado: si es de los pedidos debe llevar esa marca y, si no, ninguna. La corrección da un campo por grado con las notas que faltan y las que sobran, y un aro de acierto, error o falta sobre cada nota.
- Ordenar patrones baraja las cinco tarjetas sin dejarlas ordenadas. Se tocan de grave a agudo; tocar una colocada la quita. Se corrige puesto a puesto y la solución se muestra en color y con trastes.

11 Preguntas y corrección

Diseño acordado: fácil elige nombre con cuatro botones; medio añade inversión en un segundo desplegable; difícil añade colocar grados sobre círculos.

Opciones únicas y una correcta. En tests, fixture o semilla reproducible. El banco difícil es fijo con alteraciones y distractores; ofrecer solo los grados correctos daría una pista.

Corrección por acorde, inversión y cada destino. Acierto completo exige todos los campos del nivel. Registrar una sola respuesta por primer intento y bloquear cambios al verificar.

Rutas (DR-005). La sesión añade dos fases, `ruta` y `ruta-verificada`. Una pregunta identifica la ruta y el paso: la partida es el paso anterior de la ruta correcta (el II para llegar al V, el V para llegar al I) y las cuatro candidatas son el mismo paso de las cuatro rutas del set y modo, una por inversión, de modo que son siempre de la misma calidad y posiciones físicas distintas. Solo una es la de nuestra ruta; las demás son posiciones válidas que el material no usa aquí, y así se dice: «No es la de nuestra ruta» y «Es otra posición válida de G7», nunca «incorrecta». La candidata que se ve es la respuesta (empieza en la 1), cambiar de candidata no cambia la partida ni verifica, y Verificar compara con la ruta exacta y bloquea el selector, que sigue visible: la elegida va en rojo con □ si no es la de nuestra ruta y la de nuestra ruta en verde con ✓ (si coinciden, solo la elegida en verde con ✓), con `aria-pressed` solo en la elegida y el estado en la descripción accesible; antes de verificar no hay marcas. El rojo indica que no coincide con la ruta, no que sea musicalmente inválida. El tramo «II → V → I» encadena: tras el V (acertado o no) sigue el V → I de la misma ruta partiendo del V correcto y no del elegido, y después cambia de ruta. Solo se ofrecen Drop 2 sets 1 y 2, que son los que tienen rutas.

La tarjeta de ruta usa la ventana de trastes de su posición física, igual que en reconocimiento, y no la ventana común de la ruta que usa la plantilla: con esa ventana la candidata correcta sería la única con la numeración de trastes de la partida, lo que delataría la respuesta. Tras verificar, «Tu respuesta» y «Solución» se muestran de una en una, de modo que en móvil nunca hay más de dos diagramas a la vez.

Implementado en DR-003 y DR-004: fácil exige solo el acorde, medio acorde e inversión y difícil, además, los cuatro grados. La inversión correcta sale del bajo físico de la forma y no de su etiqueta. En difícil el banco es fijo, igual para todas las preguntas y con ocho grados únicos (1, 3, ♭3, 5, ♭5, 6, 7 y ♭7), de modo que sobran cuatro y no delata la solución; cada grado se compara tal cual con la nota de su círculo, así que ♭3 no es 3. Arrastre, dos toques, teclado y la alternativa «Colocar sin arrastrar» (cuatro botones de 48 px o más, identificados por cuerda y traste, que reflejan el estado de los círculos) envían las mismas acciones a la sesión. Con el dedo, la ficha que se ve en el arrastre va 64 px por encima del punto de contacto para que el dedo no la tape; el destino se calcula con las coordenadas reales del dedo. Tras verificar un error en fácil, el botón elegido se marca en rojo con □ y el correcto en verde con ✓, con aria-pressed solo en el elegido y los cuatro botones bloqueados; antes de verificar no hay ninguna marca. La corrección da un resultado por campo (acorde, inversión y cada círculo) con texto, símbolo y color, conserva los aciertos parciales y muestra la solución con la tarjeta de colores.

Desde 1.0.1, `voicingOf` deriva las nueve variantes de sexta mediante `src/music/sourceSixth.ts`, compartido con estudio. Se pregunta la posición con 6 y se conserva la séptima gris solo para la solución. La opción fija «Cmaj7 / C6» agrupa ambas variantes sin revelar cuál se dibuja. La clave de familia y el progreso siguen siendo `maj7`. `evaluate` compara los grados activos y `inversionFromBass` usa 0,4,7,9 en las formas C6. El banco de Difícil contiene siempre 6 y 7; no se acepta uno por el otro. Los JSON originales no cambian.

La revisión 1.0.2 indica «Tónica: C» antes de responder, en los tres niveles. No marca su ubicación en el diagrama. Tras verificar, `src/music/tonicReading.ts` identifica la raíz

mediante afinación y traste, sin depender de las etiquetas de grado ni del orden de las notas. Solo para las notas de sexta mayor o menor séptima explica la lectura equivalente: C6/Am7 o Cm7/E♭6, con la inversión obtenida del bajo físico respecto a la raíz alternativa. No añade respuestas equivalentes aceptadas: la corrección sigue referida a la tónica anunciada. La autoría «Creado por Jesús Quevedo Rodríguez» se muestra en un pie común después de la pantalla, sin posición fija ni superposición de controles. No cambia el catálogo, las rutas, el esquema de progreso ni la selección de preguntas.

DR-012 añade al modo estudio II–V–I el mismo traste ancla de la plantilla: el traste donde cae la raíz del II se sombrea en las tres tarjetas y se desplaza al cambiar de tonalidad. Se deriva en memoria para no alterar la fuente de rutas.

DR-011 añade historial a reconocimiento, igual que el test de rutas: cada intento guarda pregunta, respuesta y resultado. «Anterior» recupera el estado guardado; «Siguiente» vuelve a preguntas visitadas antes de sortear otra. «Terminar y revisar» cierra la sesión libre en la pregunta actual y lista por campo qué se acertó, qué falló y cuál era la solución, con el diagrama físico desplegable. Las preguntas no verificadas quedan identificadas como pendientes y muestran la comparación al entrar en revisión. Retroceder y revisar no vuelven a contar intentos en el progreso.

12 Móvil y accesibilidad

Al verificar reconocimiento o rutas, `useGameFocus` conserva el foco accesible en el título del resultado y desplaza la página solo lo necesario para mostrar la corrección completa y «Siguiente». Respeta el viewport visual y el margen inferior de seguridad. Si la corrección supera la altura disponible, empieza por el título y permite leer el resto desplazándose, sin reducir la letra. El juego desactiva el anclaje automático del navegador para que no deshaga este desplazamiento al sustituir la pregunta por la corrección. La pregunta siguiente vuelve a enfocar su título.

El fondo es el tramo corto de mástil de las tarjetas, no un mástil completo de doce trastes. En rutas, partida arriba, candidata debajo y selector 1/2/3/4; no cuatro candidatas pequeñas simultáneas.

Pregunta con círculos grises y fondo común. No revelar solución mediante nombre, título, tooltip o nombre accesible de las opciones. Al corregir, recuperar colores musicales y usar texto/símbolos además de rojo/verde.

Arrastre con tolerancia, alternativa de tocar ficha y destino y operación por teclado. Áreas táctiles mayores que los círculos visibles. Las tarjetas ya se miden en un E2E a 360, 390 y 430 px, y los controles de la pantalla de revisión superan los 48 px; el juego de reconocimiento, con su arrastre, se mide a esos anchos en E2E (sin desplazamiento horizontal, tarjeta completa, fichas, botones y desplegables de 48 px o más y destinos mayores que el círculo y sin solaparse); las rutas se miden igual a esos tres anchos, también con el selector bloqueado tras verificar.

13 Offline e instalación

DR-006 añade una PWA instalable con caché de recursos propios y recarga sin conexión tras una primera carga completa. No añade dependencias: el service worker, el generador y el plugin son código propio sin paquetes.

Manifest. `public/manifest.webmanifest`: nombre y nombre corto «Drop Route», idioma `es`, `display: standalone`, colores de fondo y tema `#0B1D2E`, e `id`, `start_url` y `scope` relativos (`.

`/`) para que valga igual en la raíz y en una subruta. Iconos: 192 y 512 «any» y 512 maskable, en `icons/`. `index.html` lo enlaza y declara el color del tema.

Service worker. `scripts/pwa/sw.template.js` es una plantilla; al terminar el build, `scripts/lib/pwa-build.mjs` (llamado por `scripts/pwa-plugin.mjs`) lista los ficheros del build, calcula la versión y escribe `dist/sw.js`.

- Precachea solo extensiones de recursos propios (html, js, css, woff2, png, svg, webmanifest y json) y excluye `references/`, los PDF, los mapas de código y el propio `sw.js`. Las guías PDF y la plantilla original no están en el build.
- La versión es un hash de 16 caracteres del contenido de cada recurso y del texto de la plantilla del worker. Mismo contenido y mismo worker dan la misma versión; cambiar un recurso o solo la lógica del worker da otra. No depende de fechas.
- La caché se llama `drop-route-precache:<ámbito>:<versión>`. El ámbito es la ruta de la base, de modo que las instalaciones en `/` y en `/Drop_Route/` de un mismo origen no se borran las cachés. Un test comprueba que el generador y el worker usan la misma fórmula.
- Instalar descarga cada recurso saltándose la caché HTTP. Si uno falla, borra su caché y la instalación falla: no deja una caché a medias ni toca la del worker activo.
- Una versión nueva se instala y espera. Solo la activa un mensaje `SKIP_WAITING`. Al activarse borra únicamente las cachés propias de otras versiones en su ámbito y toma el control.
- Responde solo a GET del mismo origen y dentro de su ámbito. La navegación a la raíz recibe el HTML precacheado; lo demás, su recurso precacheado o la red. Lo no precacheado no se guarda y, sin red, falla en lugar de inventar una respuesta. No usa `localStorage`: actualizar no toca el progreso.

Registro y actualización. `startPwa` (`src/ui/pwa/register.ts`) registra el worker solo en el build (no con `npm run dev`) y si el navegador lo admite; si el registro falla, la aplicación sigue sin worker. Hay versión nueva cuando una espera y ya hay otra controlando la página; la primera instalación no cuenta. La página no activa nada por su cuenta. La persona pide actualizar desde el aviso del menú: se envía `SKIP_WAITING`, y cuando el nuevo worker toma el control la página se recarga una sola vez. Con una pregunta abierta el aviso no se muestra, aunque la versión ya esté lista; vuelve a aparecer al salir al menú. Al volver la aplicación a primer plano se comprueba si hay versión nueva.

Aviso de pérdida. Si lo que hay en la página solo está en memoria (véase la tabla de estados de almacenamiento), el aviso de actualización lo dice y el botón pasa a «Actualizar y perder lo no guardado». Con el progreso guardado, actualizar conserva configuración y contadores.

Alcance del progreso. Por dispositivo y navegador, sin sincronización, cuentas, backend o notificaciones.

Lo que no está comprobado. La instalación como app en un dispositivo real (Android, iOS) y otros navegadores distintos de Chromium. Las pruebas usan Chromium de Playwright, y una de ellas sirve dos builds con un servidor propio para comprobar la actualización.

14 Documentación y Claude

`CLAUDE.md` importa `AGENTS.md` y dirige al contrato y a la orden solicitada. Las reglas permanentes están versionadas. Las tareas concretas se describen en DR-001 a DR-007 y se ejecutan por separado.

Se adapta el flujo de Mástil Escalas: rama por objetivo, pruebas dirigidas durante ajustes, revisión funcional y batería final aplicable. Un cambio material de diseño actualiza el DTS principal en Markdown durante cada fase. Los derivados y manuales completos se generan y revisan en DR-007; esta planificación no aplaza pruebas ni auditorías.

La preparación se conserva como paquete duradero. El 13/10/2026 el usuario autorizó la publicación de la base completa en el repositorio público, incluidos los originales.

15 Flujo de datos

Previsto: configuración selecciona registros; generador crea pregunta estable; tarjeta dibuja geometría; el usuario compone respuesta; evaluador calcula resultado según contrato.

En rutas, pregunta identifica ruta y paso. Cambiar candidata no cambia la partida. El estado de selección no altera los datos musicales.

La sesión y el núcleo musical se prueban sin DOM; los componentes consumen el catálogo sin duplicar posiciones. ESLint hace cumplir esta separación y Vitest se ejecuta en el entorno `node`, sin DOM.

16 Pruebas y resultados reales

La reconstrucción ejecutó 24 tests con node:test: todos pasan, sin omitidos. Verifican catálogo completo, rutas, hashes de fuentes y rechazo de errores deliberados. DR-001 los conserva sin cambios en `tests/music-catalog.test.mjs` y añade en `tests/tooling.test.mjs` 11 tests de utillaje: el lanzador de Python y que el workflow y la configuración de Playwright conservan sus pasos. El calendario documental añade además 9 tests de política documental en `tests/documentation-policy.test.mjs`. DR-002 añade 7 tests del icono en `tests/icon.test.mjs`; con ellos, node:test sumaba 51 al terminar DR-002. Las fases posteriores amplían estas cifras, que se conservan solo como historia: el recuento de cada fase está en su registro de validación. `npm test` ejecuta primero node:test y después Vitest, de modo que ninguno queda fuera. Las dos instantáneas históricas de la base inicial, descritas en la estructura del proyecto, conservan sus cifras de entonces y ningún test las comprueba.

La auditoría pasa con 124 posiciones (112 evaluables y doce de referencia) y 16 rutas. Hay oráculos independientes para D2S1 mayor primera inversión y D2S2 menor fundamental.

Se prueba también una forma G7 válida en otra inversión que no cumple la ruta. Los tests no se limitan a comparar las tablas consigo mismas.

DR-001 añade 15 tests Vitest en cinco ficheros y 10 ejecuciones E2E (cinco escenarios en dos proyectos). Vitest comprueba las capas sin DOM: las 124 posiciones y los 48 pasos de ruta del catálogo devuelven por bajo real la inversión que declaran, el bajo desplazado un semitono se detecta y el orden del array no influye. También comprueba el resumen de datos y la pantalla de arranque, y que la configuración amplía las exclusiones predeterminadas con e2e, tests y los worktrees. Playwright prueba el build servido por vite preview en un proyecto de escritorio y otro móvil: título, estado, datos empaquetados, que dist/ se generó durante esa ejecución y que el preview sirve exactamente su index.html, ausencia de errores de consola y de red, y falta de desplazamiento horizontal a 360 px.

DR-002 eleva Vitest a 50 tests en ocho ficheros y los E2E a 27 ejecuciones, sin omitidos. Vitest comprueba, para las 112 formas, que cada círculo queda centrado entre dos trastes y sobre su

cuerda, que cabe dentro del mástil y del lienzo, que no se solapa, que hay dos cuerdas marcadas con ×, que la numeración y el cejuelo son los esperados y que las cuatro calidades de un set e inversión comparten ventana, incluida la de Drop 2+4 set 1, 3ª inversión, que empieza en el traste 12 como la plantilla sin añadir C6. Verifica también tres oráculos leídos del DOM de la plantilla. La vista neutra se comprueba en las 112 formas: solo usa sus grises, solo contiene números como texto, no lleva identificadores ni títulos y su nombre accesible lista únicamente posiciones. Una prueba demuestra que depende solo de las posiciones. La solución se comprueba contra el color y la etiqueta de cada círculo. En los E2E, las tarjetas se miden sobre el DOM renderizado a 360, 390 y 430 px, en neutra y solución, en los siete sets y en los dos proyectos. Otro E2E abre la plantilla original en el navegador y compara sus 112 tarjetas con las de la aplicación en ventana (sin excepciones), posición de cada círculo, cejuelo, marcas, nombre, familia, colores y etiquetas. Los tests del icono decodifican su PNG de 32 px y miden color, contraste y separación de los cuatro círculos.

DR-003 y DR-004 elevan Vitest a 131 tests en trece ficheros y los E2E a 93 ejecuciones, sin omitidos. DR-005 los eleva a 163 tests y 123 ejecuciones. Vitest comprueba con semilla y fixtures: opciones únicas y permutación de las cuatro calidades evaluables, ausencia de repetición inmediata en los siete sets, repetición admitida solo con una forma disponible, respuesta completa por nivel, corrección por campos con la inversión sacada del bajo físico (fixture con la etiqueta mal declarada y las notas desordenadas), las 112 formas corregidas como correctas, el banco fijo de siete grados sin duplicados, las acciones de colocar, mover, intercambiar, devolver, cancelar y quitar con 200 secuencias aleatorias que conservan sus invariantes, la comparación de b3, b5 y b7 con 3, 5 y 7, y las pantallas con sus estados y sin pistas. Los E2E recorren el juego en el build: fácil correcto e incorrecto, medio con inversión fallida, difícil por dos toques, arrastre con ratón, arrastre táctil con eventos reales de Chromium, pointercancel, soltar fuera, cambio, devolución, cancelación, teclado, aciertos parciales, Siguiente, Salir, los siete sets y 360, 390 y 430 px. La respuesta correcta se deduce en el E2E de las posiciones visibles y de `data/catalog.json`, sin semilla de pruebas ni atajos en la aplicación.

DR-005 añade a Vitest, con semilla y con el documento real de rutas más un fixture: dieciséis rutas y cuatro por set y modo; solo Drop 2 sets 1 y 2 en rutas; cuatro candidatas únicas, de la misma calidad, con posiciones físicas e inversiones distintas, para los dos sets, los dos modos y los tres tramos; el I menor siempre Cm7; la respuesta contra una tabla independiente de inversiones (V: 2ª, 3ª, fundamental y 1ª; I: la inicial); no repetir ruta y que salgan las 16; partida estable al cambiar de candidata; verificación única con bloqueo; el encadenado II → V → I partiendo del V correcto tras fallar; y las pantallas, con su neutralidad y la redacción sin «incorrecto». Los E2E deducen la candidata correcta de las posiciones de la partida y de `data/routes.json`, y cubren los dos modos, los tres tramos, el deslizamiento con ratón y con eventos táctiles, el teclado y los anchos de 360, 390 y 430 px, con un máximo de dos diagramas visibles.

DR-006 añade pruebas en tres niveles; las cifras observadas están en su registro.

- Vitest, con almacenamiento simulado: el conteo por transición (cada verificación una vez, acierto completo, rutas por tramo, «II → V → I» por mitades, ver la corrección sin contar), el esquema y la migración desde el fixture 0, la reparación contador a contador, los casos de la tabla de estados (dañado, reparado, futuro, bloqueado, lleno y copia fallida, con el original intacto), el resumen, los límites de capas, el registro del service worker, el adaptador del navegador y las pantallas del panel y los avisos.
- node:test sin navegador: el generador (base de publicación, qué se precachea, versión por contenido y por plantilla, nombre de caché), el worker con un arnés que simula Cache API y red, un `npm run build` real con base `/` y `/Drop_Route/`, y los iconos de la PWA (maskable opaco con el dibujo dentro de la zona segura, 192 con los cuatro círculos y `public/icons/` idéntico a las exportaciones).

- Playwright, con Chromium real y sin mocks de service worker ni caché: `progress.spec.ts` (contadores, recarga y los casos de almacenamiento, incluidos copia fallida y migración), `offline.spec.ts` (primera carga, recarga y juego sin conexión, tarjetas, fuentes, manifest e iconos desde el worker) y `pwa-update.spec.ts` (dos builds A y B en un servidor propio: B en espera con una pregunta abierta, oferta solo en el menú, una única recarga, progreso conservado, cachés ajenas y de otra base intactas, almacenamiento lleno y bloqueado).

DR-013 añade pruebas en tres niveles; las cifras están en su registro.

- node:test: la validación de los 20 patrones de escala, con tres oráculos y nueve mutaciones deliberadas.
- Vitest: las doce tonalidades de las cuatro escalas, el orden de grave a agudo y la tarjeta. En la sesión, la rotación, las respuestas completas, la corrección de marcas y del orden, el historial y la configuración. En el progreso, el conteo, los contadores, la migración 1 → 2 y el almacén. En las pantallas, la neutralidad, la corrección y que al ordenar no se revelen los trastes.
- Playwright: `scales.spec.ts` (estudio, los tres niveles de patrones, ordenar y 360/390/430 px) y la migración 1 → 2 en `progress.spec.ts`. Deducen la respuesta de las posiciones dibujadas y de `data/scales.json`.

Controles negativos de DR-006 observados por el supervisor: eliminar el hash de la plantilla del worker del cálculo de la versión hizo fallar 1 test, y eliminar el ámbito del nombre de la caché hizo fallar 2; los ficheros se restauraron byte a byte. Los E2E nuevos no tienen todavía resultado registrado.

Los controles principales de fases anteriores se probaron con mutaciones deliberadas: círculos desplazados 3 px, un dígito de color cambiado y círculos del icono encogidos. Cada una hizo fallar el test correspondiente.

El lint y las reglas de capas se probaron con importaciones prohibidas deliberadas, y las exclusiones de Vitest con tests que fallarían si se ejecutaran. Esas pruebas fueron manuales y no forman parte de la suite. Los resultados concretos de DR-001 están en su registro de validación.

17 Construcción y publicación

npm run docs:build regenera LaTeX/PDF desde el Markdown. Guarda hashes del Markdown, derivados, plantilla y generador para detectar desactualizaciones.

docs/documentation-policy.json mantiene dtsDerivatives en deferred durante DR-001 a DR-006. npm run docs:check valida enlaces, órdenes, política, manifiesto y archivos existentes. Los cambios del Markdown, plantilla o generador se declaran pendientes sin obligar a recompilar. La ausencia o alteración de los derivados conservados sigue siendo un error.

npm run docs:check:final exige que todos los hashes coincidan, con independencia de esa política. DR-007 genera y revisa las páginas, termina los manuales y cambia dtsDerivatives a required, de modo que la comprobación ordinaria también exige sincronización. La CI de cierre incorpora el comando estricto.

La CI preparada tiene dos trabajos con `npm ci --ignore-scripts`. El de la base usa Node 24 y Python 3.10, el mínimo admitido, y ejecuta data:check, audit:music, test:base y docs:check. El de la aplicación ejecuta typecheck, lint, test:unit, build, la descarga de Chromium con sus dependencias del sistema y test:e2e. Un test de node:test comprueba que el workflow conserva esos pasos. Los resultados de cada entrega están en su registro de validación y en Actions; para

DR-006 todavía no hay una ejecución registrada. El código en Git tampoco equivale a tener la aplicación desplegada.

Base de publicación. Sin configuración, `vite build` publica en la raíz (`/`). `DROP_ROUTE_BASE` fija otra base; la forma recomendada es `/Drop_Route/`, con barra inicial y final. `normalizeBase` también admite `Drop_Route` y `/Drop_Route` y los convierte a `/Drop_Route/`; rechaza con un mensaje claro las URLs y las rutas inseguras, como `./`, `../`, espacios o barras dobles. Por ejemplo, `DROP_ROUTE_BASE=/Drop_Route/ npm run build` en bash, o `$env:DROP_ROUTE_BASE = '/Drop_Route/'; npm run build` en PowerShell. Algunos shells de Windows convierten las rutas que empiezan por `/`; en ese caso el build falla en lugar de publicar mal. Se sirve el contenido de `dist/` en esa subruta, por HTTPS o desde `localhost`, que es lo que exigen los service workers. La base cambia el ámbito del worker, el nombre de sus cachés y las URL de los recursos; el manifest, al ser relativo, no cambia. Un test construye de verdad con `/` y con `/Drop_Route/` y comprueba el worker resultante. No existe todavía un flujo de despliegue ni se ha elegido el alojamiento.

Comando	Función
`npm run dev`	Servidor de desarrollo de Vite.
`npm run build`	Comprueba tipos y genera `dist/`, con `sw.js`. Admite `DROP_ROUTE_BASE`.
`npm run preview`	Sirve `dist/`.
`npm run typecheck`	`tsc` sobre aplicación y herramientas.
`npm run lint`	ESLint, con los límites entre capas.
`npm test`	node:test de la base y después Vitest.
`npm run test:e2e`	Playwright sobre un build recién generado.
`npm run icon:export`	Regenera las exportaciones del icono y de la PWA desde sus SVG (Chromium). `-- --solo-pwa` rehace solo el 192 y el maskable.

Playwright genera el build y lo sirve con vite preview al ejecutar los E2E. Nunca reutiliza un servidor existente: con el puerto 4173 ocupado, la ejecución falla antes de probar nada. El humo comprueba además que dist/index.html es posterior al inicio de la ejecución y que el preview sirve ese mismo fichero, de modo que no se prueba una compilación anterior ni el servidor de desarrollo. Ningún preview queda iniciado. Los scripts `test:unit` y `test:base` ejecutan cada corredor por separado.

El comando docs:build funciona con Pandoc 3.12.1 y LuaLaTeX. El script y la plantilla se adaptaron en DR-001. LaTeX falla con las rutas temporales cortas de Windows, que contienen una tilde. Pandoc 3.12 envuelve las tablas con un contador none que la plantilla no definía, y ya no emite hypertarget en los títulos. Además, `.gitattributes` fija ahora LF para los `.tex`, porque en Windows la copia con CRLF cambiaba los hashes del registro.

El primer push fue bloqueado por revisión automática debido al repositorio público y los materiales originales. El 13/10/2026 se obtuvo autorización explícita del usuario para publicar la base completa. Las futuras publicaciones siguen requiriendo una orden que las cubra.

18 Limitaciones actuales

- El progreso se guarda por dispositivo y navegador: no se sincroniza ni se exporta, y las copias de seguridad de datos dañados, reparados o migrados no se pueden recuperar desde la interfaz. Hay solo contadores, sin puntuación detallada, rachas ni repaso. Las rutas existen solo para Drop 2 sets 1 y 2, mayor y menor; no se ofrecen otras familias porque faltan sus rutas normalizadas. Las pruebas móviles usan emulación de Chromium, también el arrastre táctil, que se probó con eventos táctiles de Chromium y no con un dedo real ni con Safari.
- En difícil, el área táctil de cada destino sobre la tarjeta es la celda del círculo (unos 55 por 25 px a 360 px): mayor que el círculo, pero menor que 48 px de alto porque las cuerdas están a 20 unidades y áreas mayores se solaparían. Por eso existe «Colocar sin arrastrar», con cuatro botones de 48 px o más. Las fichas, botones y desplegables también superan los 48 px.
- La tarjeta de ruta no usa la ventana común de ruta de la plantilla, por no delatar la respuesta (véase arriba). El deslizamiento es opcional: el selector 1 a 4 y las flechas hacen lo mismo.
- La puntuación detallada sigue pendiente de decisión; los contadores de DR-006 cuentan intentos y aciertos sin más.
- El icono está aprobado. El PNG de 192 y el maskable de 512 los generó el supervisor con CairoSVG 2.9.1 y no con el Chromium del script; el 512 «any» es el aprobado.
- La PWA solo se ha probado en Chromium. No se ha comprobado la instalación en un dispositivo real, en Safari ni en Firefox, y las pruebas de actualización usan un servidor propio y no un alojamiento real.
- GitHub Pages publica la aplicación desde `main`. El paquete de esta entrega es 1.1.0; la versión visible procede de ese metadato y el esquema de progreso pasa al 2, con migración desde el 1. El estado del despliegue se comprueba en GitHub Actions.
- Reconocimiento, estudio y revisión muestran las nueve variantes C6 de la fuente sobre copias derivadas. El juego de rutas conserva los pasos canónicos; sus variantes opcionales se consultan en estudio.
- Las fuentes manuscritas pesan 87 kB y se descargan al mostrar tarjetas; hasta entonces el texto usa una fuente de respaldo.
- El LaTeX y el PDF del DTS son la última edición generada, hecha durante DR-001 antes del calendario documental. Están pendientes respecto a este Markdown hasta DR-007 y no deben presentarse como documentación actualizada.
- Playwright solo prueba Chromium; Firefox y WebKit no están configurados.
- Python se ejecutó en esta revisión solo con la versión 3.13.7. El mínimo 3.10 se comprobó por gramática y por las API usadas, y lo ejercita la CI, que usa esa versión.
- Solo se normalizaron las rutas Drop 2 sets 1 y 2 del HTML.
- Los PDF tienen 21 páginas de imágenes cada uno; no se han transcrito íntegramente.
- Cdim7 sigue como referencia. C6 se evalúa como variante de la familia mayor en reconocimiento, sin añadir una quinta opción de calidad.
- La transposición de voicings y rutas se limita al modo estudio; solo los juegos de escalas eligen tonalidad. El audio queda para otra etapa.
- Los patrones de escala son las cinco formas movibles de una única fuente, sin posición abierta, 3NPS ni otros modos. En Medio y Difícil, el área táctil de cada nota es su celda (unos 20 px de alto), como en el tablero de grados; los botones y el banco miden 48 px o más.
- La reconstrucción conserva contratos y datos, no recupera el commit local perdido byte a byte.
- Hosting y licencia del código nuevo pendientes; las referencias conservan su autoría.

19 Evolución

DR-001 configuró las herramientas; DR-002 creó tarjetas e icono; DR-003 y DR-004, el reconocimiento; DR-005, las rutas. Las cinco están integradas en main. DR-006, progreso y PWA, está implementada en feat/progress-and-pwa y pendiente de revisión. DR-007, cierre documental y pruebas, está pendiente: en ella se regenerarán LaTeX y PDF y se escribirán los manuales. DR-013 añade los patrones de escala (datos, estudio y dos juegos) y el esquema 2 del progreso.

Convertir lo planificado en descripción de implementación solo después de inspeccionar y probar el código. No extender una tarea a mejoras no pedidas.

20 Apéndice: evidencia y límites

Verificados: mismas tres referencias, SHA-256, extracción, notas físicas, fórmulas, grados, inversiones, rutas y 24 tests ejecutados nuevamente. Documentación reconstruida desde las decisiones visibles de la conversación.

Las comprobaciones de cada fase se registran en el directorio de validación del repositorio, junto con la compilación del DTS anterior, su revisión visual y la validación documental de la base. Los juegos de reconocimiento y de rutas están implementados e integrados en main; el progreso y la PWA, en la rama de DR-006, con la evidencia que conste en su registro.